

BEFORE WE BEGIN

Make sure you have installed
Git in your system.

SCAN THE QR FOR
GIT
INSTALLATION
GUIDE





KHARAGPUR OPEN SOURCE SOCIETY

GIT AND GITHUB WORKSHOP



WHO ARE WE? 🚀

Basically, we are a bunch of tech nerds united by a love for **open-source**, caffeine, and building **cool stuff**.



WHO ARE WE? 🚀

Basically, we are a bunch of tech nerds united by a love for **open-source**, caffeine, and building **cool stuff**.

We exist to help you survive the tech world. What we do:

- 🛠️ **Workshops**: Git, GitHub, Linux, Cybersecurity.
- ❄️ **KWoC**: Kharagpur Winter of Code
- 🌍 **GSoC**: Google Summer of Code Mentorship
- 🐛 Writing bugs and occasionally fixing them



>> OVERVIEW

- Terminal

- Version Control

- What is Git?

- Git Commands

- Demo-1

- Branching

- What is GitHub?

- Pull Requests

- Demo-2

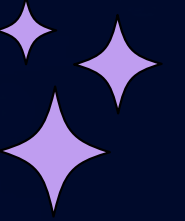
- War crimes

THE TERMINAL

➤ What is a Terminal?

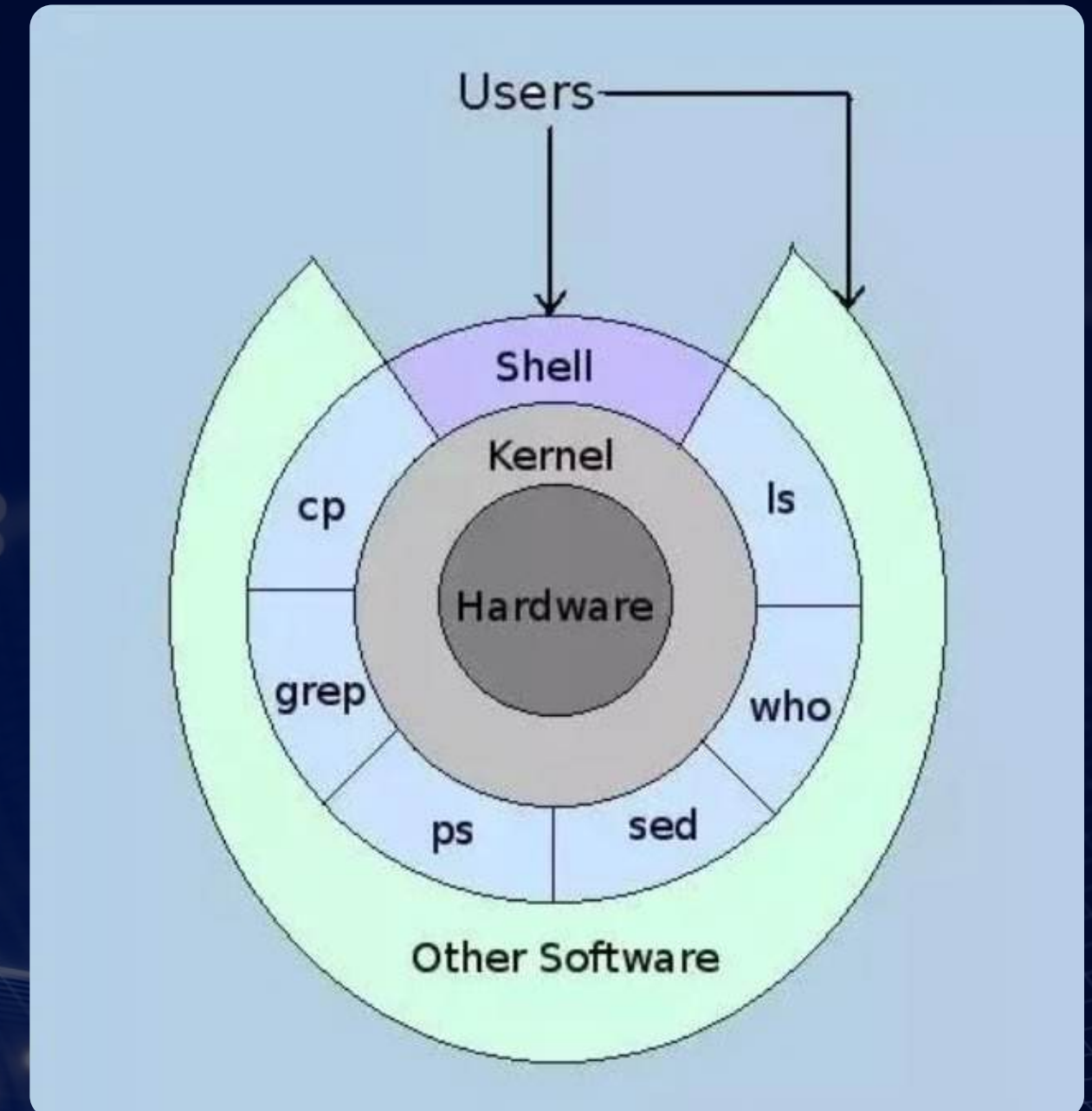
```
[ 🌀> 🏠~ _____ 🌀 impure < 11:48:39 PM IST ]  
> fortune | lolcat  
timesharing, n:  
    An access method whereby one computer abuses many people.
```

THE TERMINAL



➤ What is a Terminal?

- A text-based interface to give instructions to your computer
- Faster and more powerful than only using GUIs



TERMINAL COMMANDS



`pwd` (Print Working Directory): Tells you exactly where you are right now.

```
Path
----
C:\KOSS\Git and GitHub workshp
```

`cd <folder_name>` (Change Directory): Moves you into another folder.
(Tip: `cd ..` takes you one step backward).

```
PS C:\KOSS> cd "Git and GitHub workshp"
PS C:\KOSS\Git and GitHub workshp> |
```

TERMINAL COMMANDS



`ls` (List): Lists all the files and folders in your current location

```
PS C:\KOSS\Git and GitHub workshp> ls
```

```
Directory: C:\KOSS\Git and GitHub workshp
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	26-08-2026 15:14	0	Branching.txt
-a----	26-08-2026 15:13	0	Git.txt
-a----	26-08-2026 15:13	0	GitHub.txt
-a----	26-08-2026 15:13	0	Terminal.txt

```
PS C:\KOSS\Git and GitHub workshp> |
```

TERMINAL COMMANDS



`mkdir <folder_name>` (Make Directory): Creates a brand new folder right where you are standing.

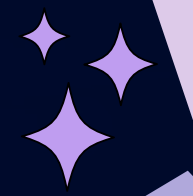
```
PS C:\KOSS\Git and GitHub workshp> mkdir "KwoC"
```

```
Directory: C:\KOSS\Git and GitHub workshp
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-----	26-08-2026 15:23		KwoC

```
PS C:\KOSS\Git and GitHub workshp> |
```

VERSION CONTROL



➤ *What is Version Control?*

- It operates exactly like a "Save Game" system for your code. You can always reload your last checkpoint!
- It records every single modification you make, letting your team safely experiment, collaborate, and rewind if everything breaks.

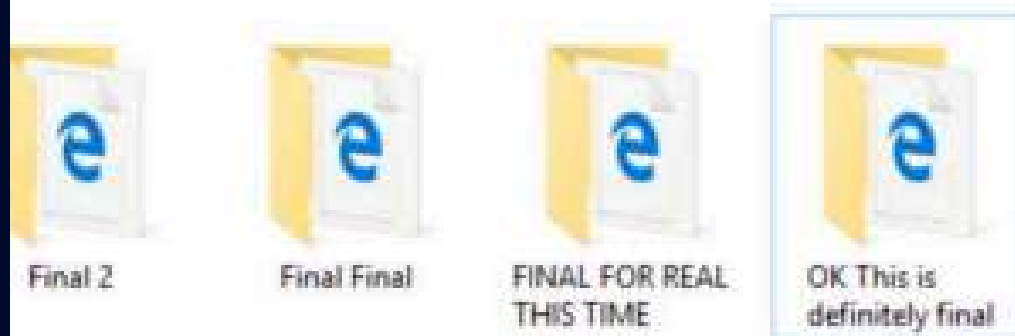




I prefer the real version control

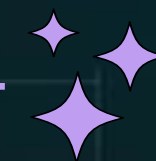


I said the *real* version control



Perfection

GIT



Git is a **free and open source** version control system, used for source code management.



Git was originally created by **Linus Torvalds**, (who also happens to be the creator of Linux) for version control in the development of the Linux kernel.



HOW LINUS TORWALDS FELT AFTER CREATING BOTH LINUX AND GIT

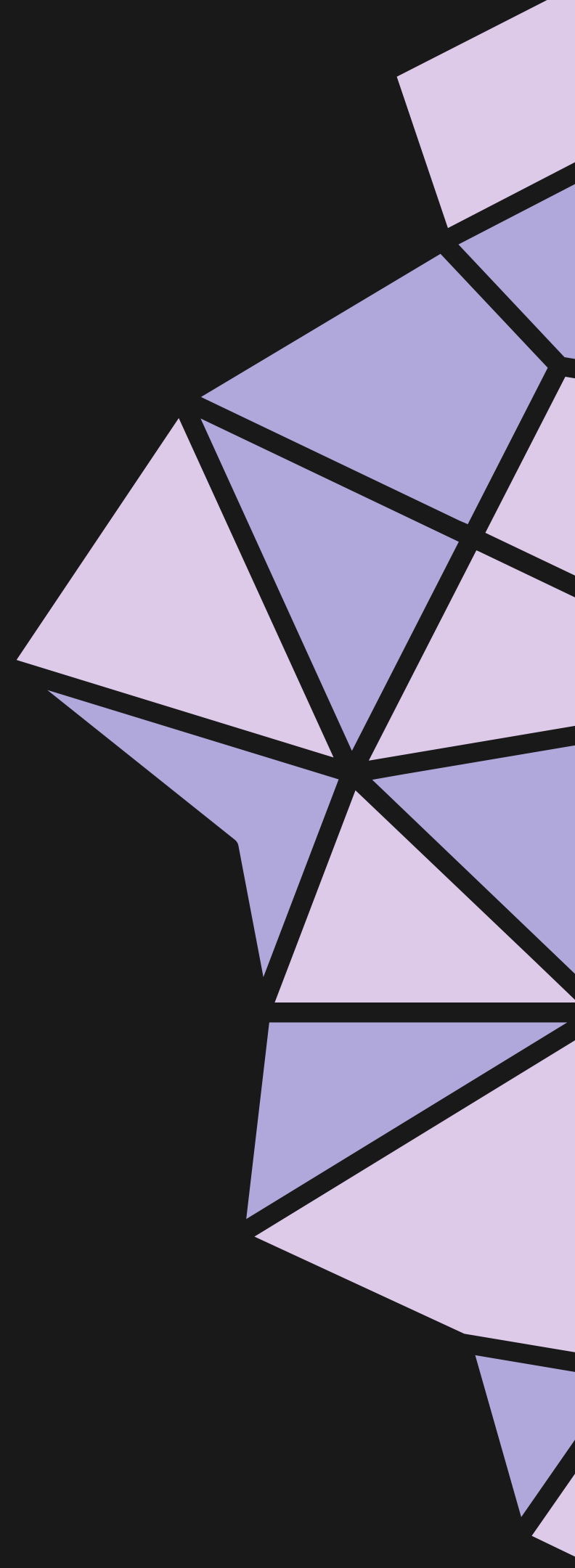


GIT STASH YOUR DOUBTS

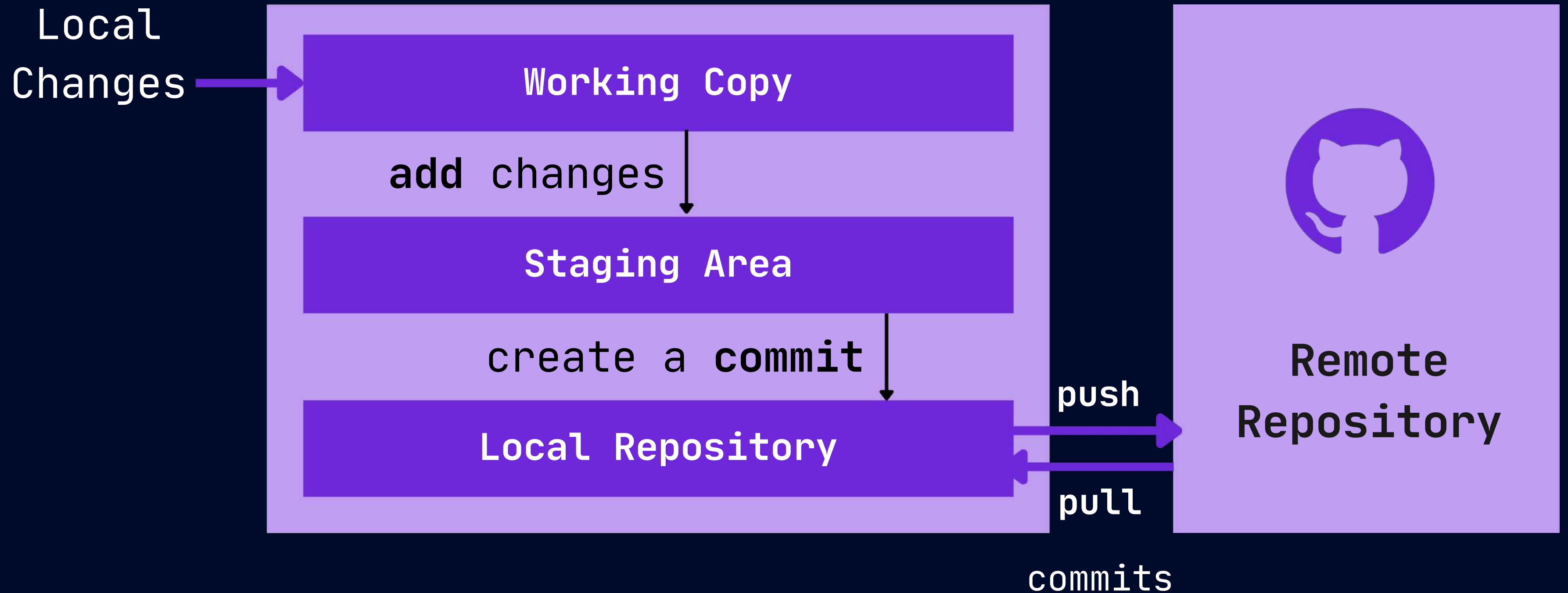
receives notification from GitHub

gets excited someone starred your repo

"Potential security vulnerability found"



GIT WORKFLOW



This Staging Area is Confusing!



WORKING
DIRECTORY



© CanStockPhoto.com - csp52904870

STAGING AREA



COMMIT !

NOW LET'S DIVE DEEP INTO GIT COMMANDS



INITIALISING A REPOSITORY



INITIALISATION:

```
$ cd dir-name  
$ git init
```

```
❯ ~/Development/builds-systems/nixos-scripts > git master ↑1  
tree -L 1 .git  
.git  
├── COMMIT_EDITMSG  
├── config  
├── description  
├── HEAD  
├── hooks  
├── index  
├── info  
├── logs  
├── objects  
└── refs
```

GIT COMMANDS



\$ `git add <path>`

adds the specified files to the staging area

\$ `git status`

shows files in current working directory and staging area

\$ `git commit -m "message"`

creates a commit object with the added changes

git commit



git push



git add .



COMMIT MESSAGE



A commit message is essentially a sticky note left for your teammates (and your future self). It should clearly explain what you changed and why. Don't leave them guessing!

Examples:

feat: add Google login button

fix: cart is persistent across different tabs

GIT COMMIT WAR CRIMES

Please, for the love of open-source, don't do this.

War Crimes

- "update" (Update what?!)
- "fixed stuff" (Which stuff?!)
- "asdfghjkl" (Ah, the keyboard smash)
- "it works but I don't know why"
- "final update" ...followed by "final final update"

ALL I HEARD WAS
'I SWEAR IT WILL BE FUNNY...'
AND THEN WE WERE IN JAIL.



Good Citizen

- feat: add dark mode toggle (Add a new feature)
- fix: resolve crash on login path (Fix a bug)
- docs: update README instruction (Change documentation)
- chore: clean up old comments (Maintenance stuff)

DOUBTS?



PS: USE GIT HELP <COMMAND_NAME>

SIDE QUEST 1

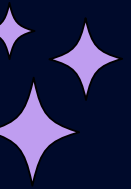
- | | |
|---|----------|
| 1. Make a new directory and move into it | 1 point |
| 2. Make two files intro.txt and user.txt | 1 point |
| 3. Add and commit the changes with a message | 3 points |
| 4. Put your username and intro in these files | 2 points |
| 5. Add and commit the changes with a message | 3 points |

PS: use git status in between commands to make sure things are working correctly, use git log to view the commit history

DEMO_gorgon



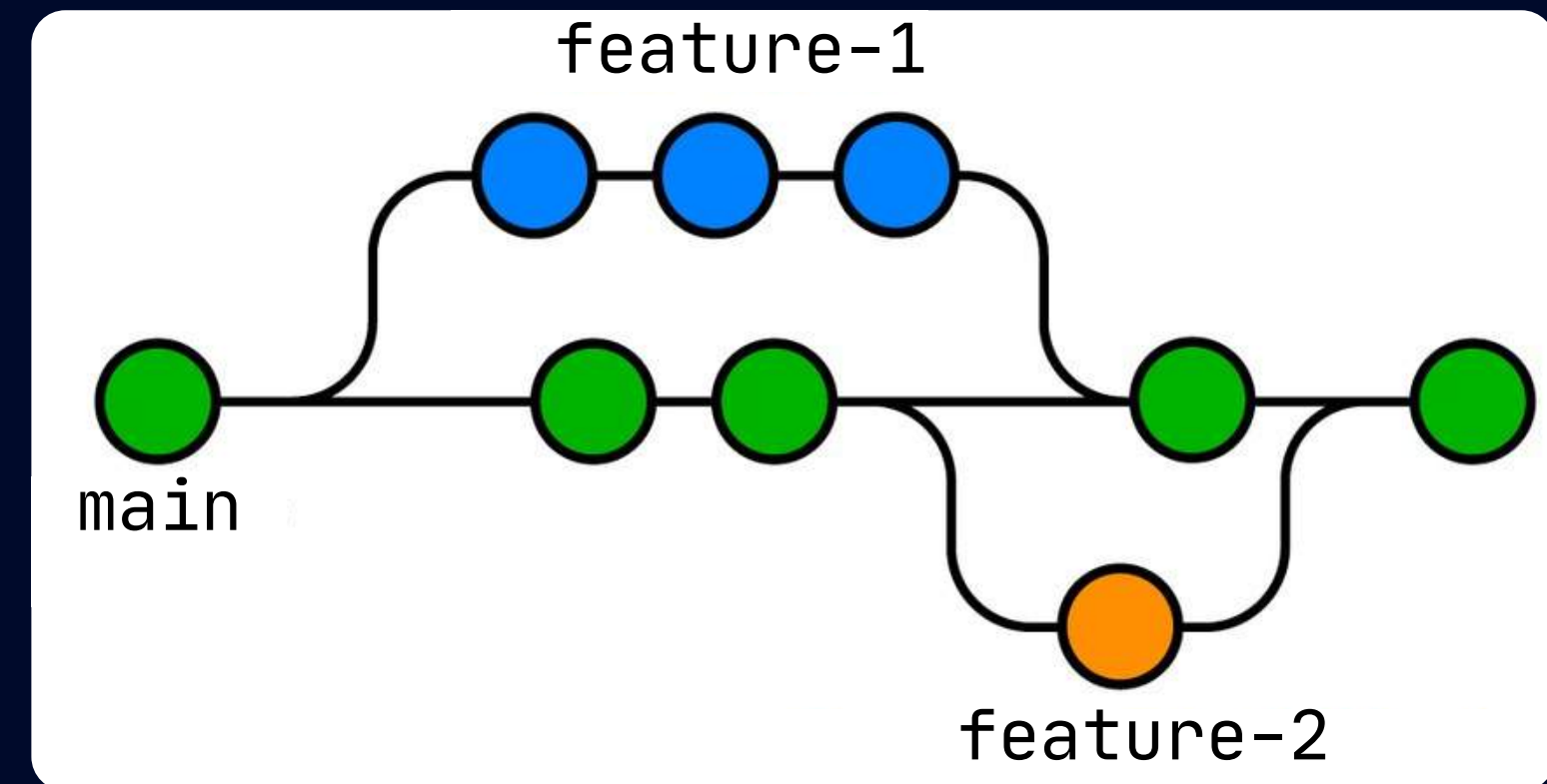
BRANCHING



In a git repository, you can create multiple '**branches**', which are different versions of the source code.

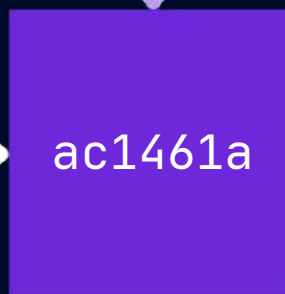
Branches allow you to work on **different tasks** in the repository **separately**, without affecting the main version of the code.

Changes made in the branches can be **combined** with the main code, when it is ready.



(main) \$

main



ac1461a

```
(main) $ git add .
```

```
(main) $ git commit -m "feat: add about us page"
```



```
(main) $ git branch auth
```

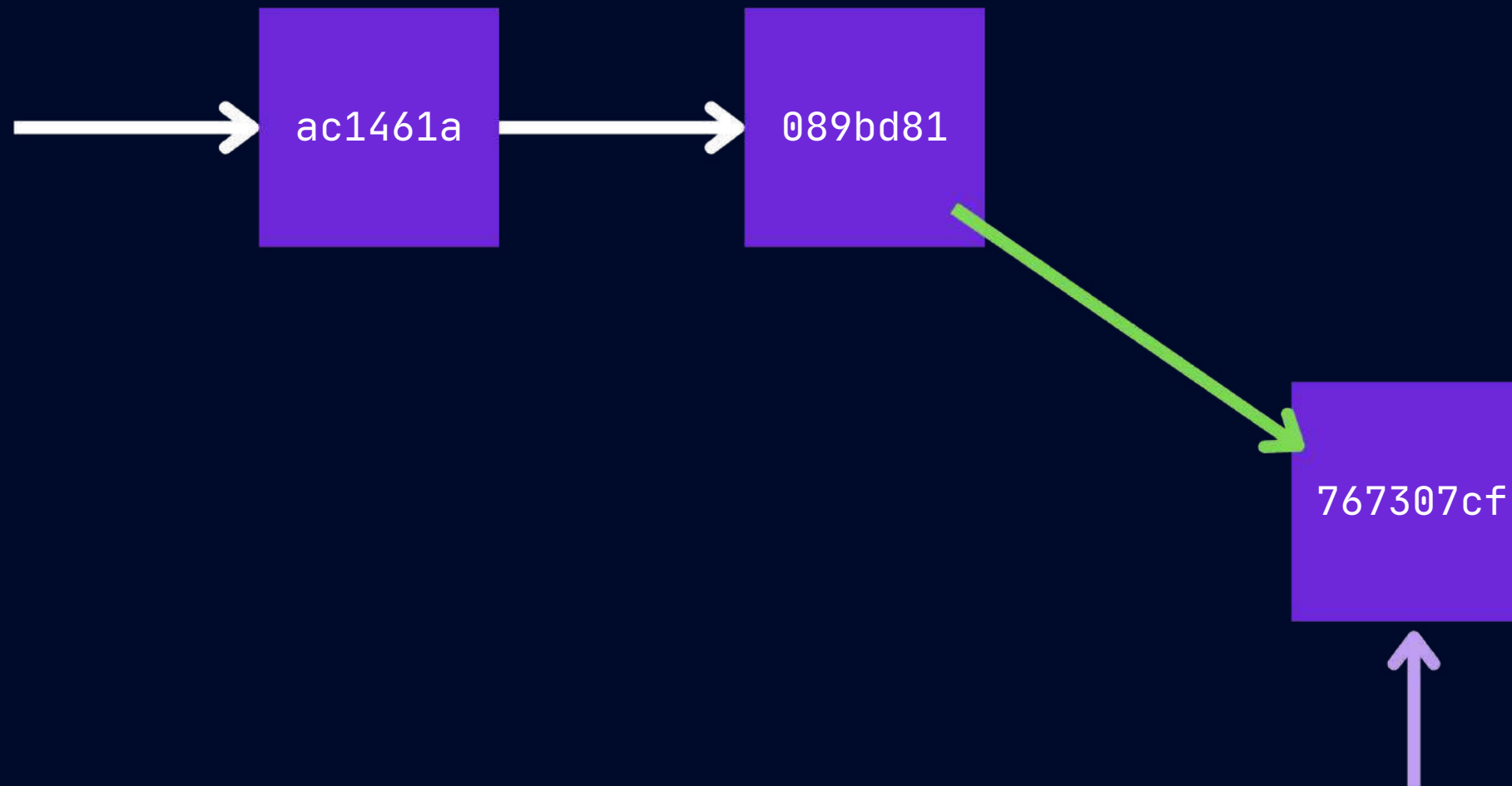
```
(main) $ git checkout auth
```



```
(auth) $ git add .
```

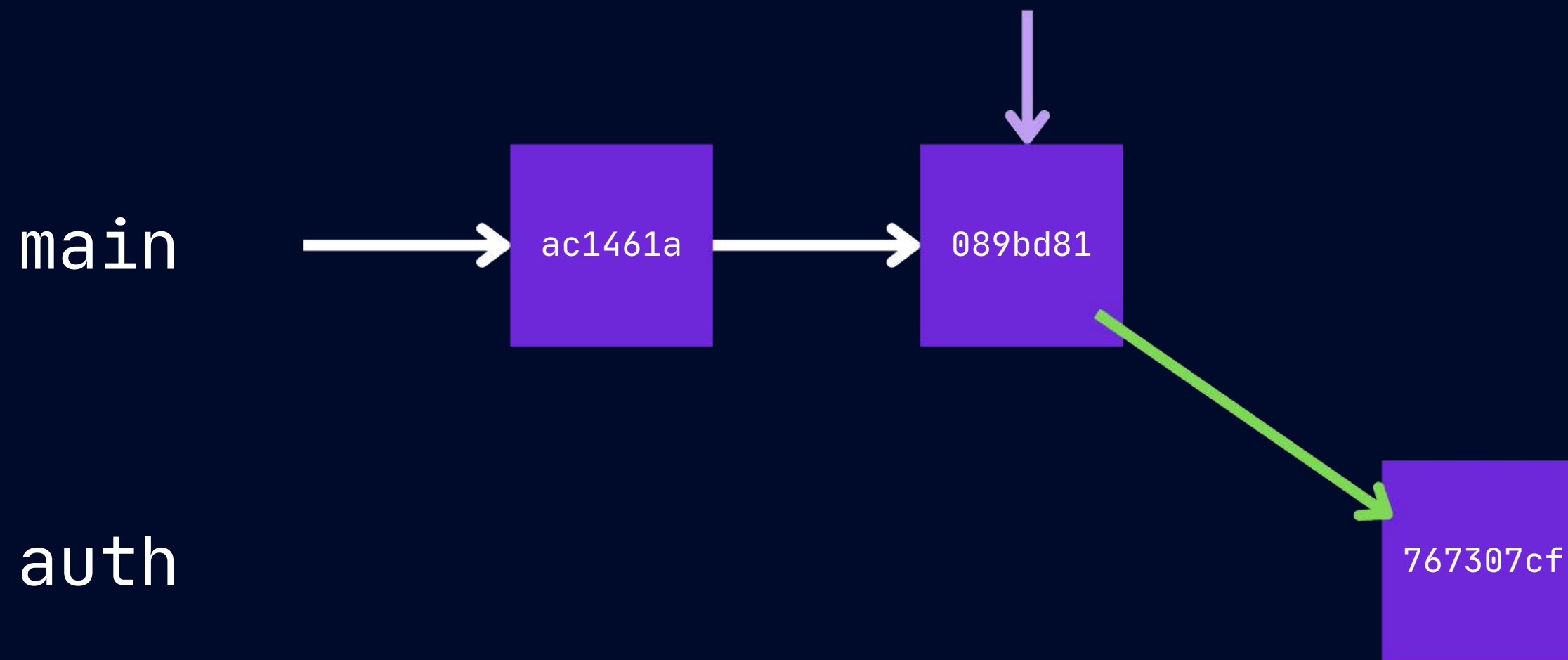
```
(auth) $ git commit -m "feat: implement login page"
```

main



auth

(auth) \$ git checkout main

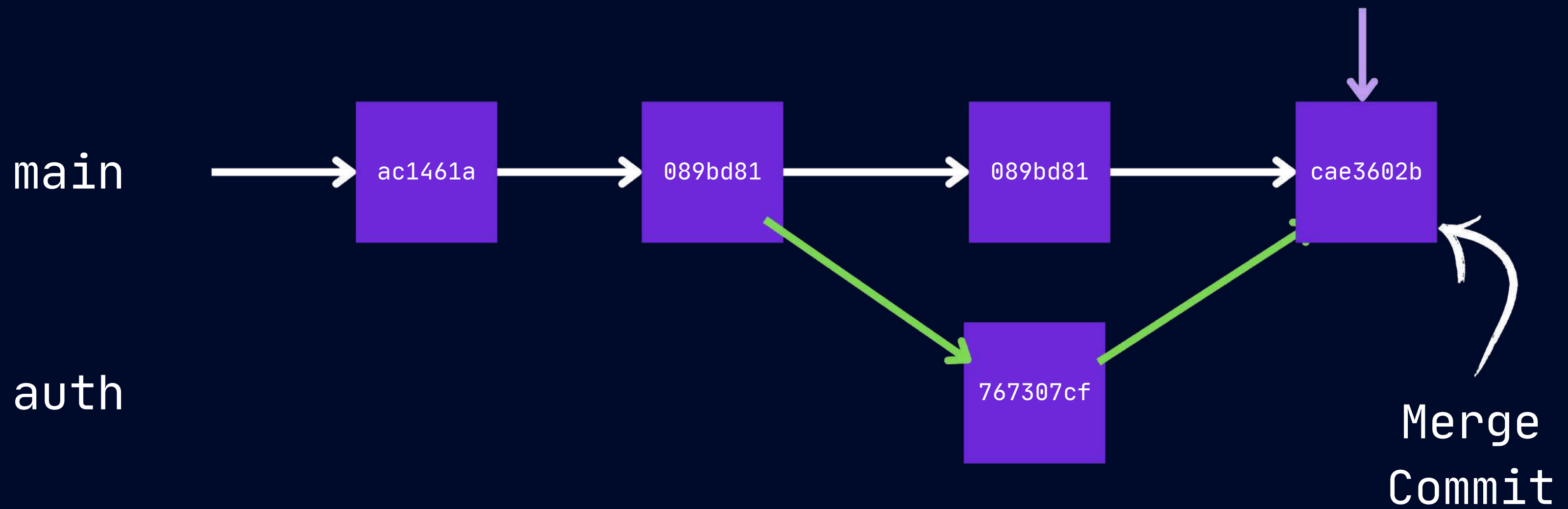


```
(main) $ git add .
```

```
(main) $ git commit -m "fix: typo"
```



```
(main) $ git merge auth
```



MERGE CONFLICTS



- The Problem: Two people edited the exact same part of a file differently. Git pauses the merge to stop you from accidentally overwriting someone's work.
- The Solution: Don't panic at the weird symbols! Editors like VS Code make it easy. Just click "Accept Current" or "Accept Incoming", save the file, and commit

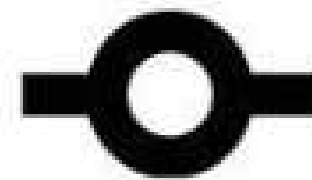
```
1 Hello, World!  
   Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes  
2 <<<<<<< HEAD (Current Change)  
3 My favourite fruit is a mango.  
4 =====  
5 My favourite fruit is an apple.  
6 >>>>>>> superior (Incoming Change)  
7 |
```



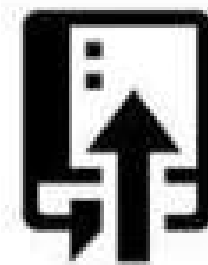
Git merge

DOUBTS

IN CASE OF FIRE



1. git commit



2. git push



3. git out!

PS: USE GIT HELP <COMMAND_NAME>

SIDE QUEST 2

- | | |
|---|----------|
| 1. Make a new branch from master | 1 point |
| 2. Make a file fav_projects.txt and
add your favorite projects | 1 point |
| 3. Add and commit the changes with a message | 2 points |
| 4. Checkout to master | 3 points |
| 5. Merge the two branches to master | 3 points |

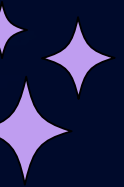
PS: use git status in between commands to make sure things are working correctly, use git log to view the commit history

GIT VS GITHUB



Le GitHub Creators

GITHUB

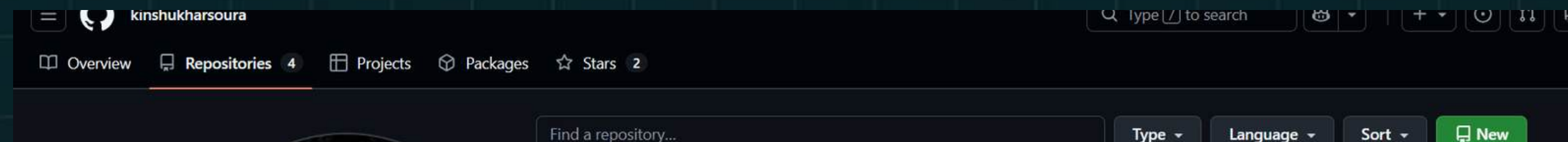
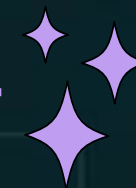


GitHub is a platform that allows users to manage their git repositories.

It is used commonly to host open source projects.

GitHub allows developers to collaborate on a project from anywhere.

CREATING A REPO



General

Owner *

kinshukharsoura

Repository name *

GSoc

✔ GSoc is available.

Great repository names are short and memorable. How about [probable-telegram?](#)

Description

For complete GSoC guidance and prep

35 / 350 characters

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Name of your repo

Click this!

CREATING A REPO



Quick setup — if you've done this kind of thing before

 Set up in Desktop

or

HTTPS

SSH

`https://github.com/dipamsen/my-repo.git`



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# my-repo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/dipamsen/my-repo.git
git push -u origin main
```



...or push an existing repository from the command line

```
git remote add origin https://github.com/dipamsen/my-repo.git
git branch -M main
git push -u origin main
```



use these commands to push an
existing git repo to GitHub

MAKE CHANGES TO A REPO

The beauty of Open Source Software is that anyone can contribute to it!

To contribute to a repo (which you don't have write-access to):

1. **Fork** the Repo to your GitHub account
2. **Clone** the fork locally
3. Make changes to your local repository (on a different branch)
4. Create a **Pull Request** from your branch to the repo.
5. Wait for the maintainers to review and merge your Pull Request! :)

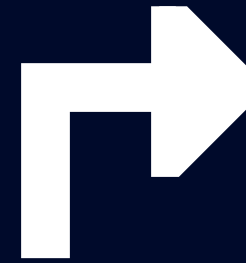
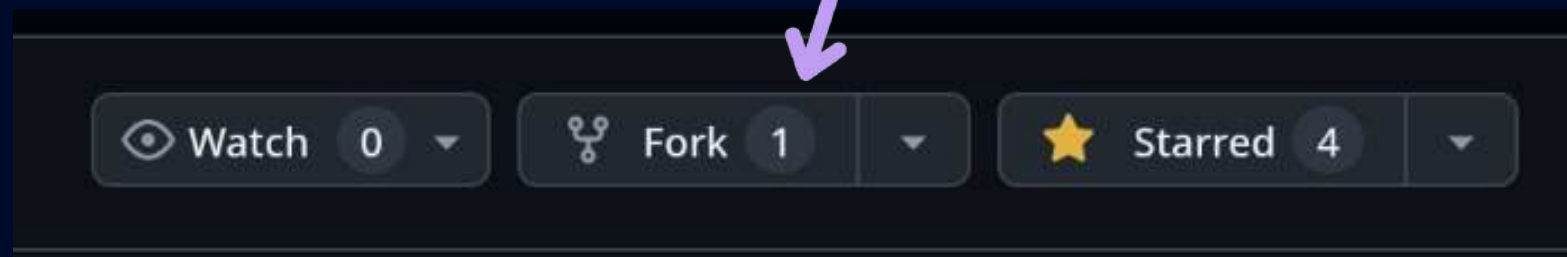
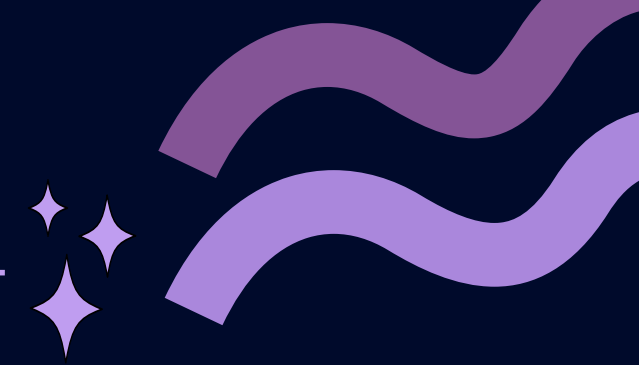
ISSUES



- Issues can represent a wide range of tasks, including bug reports, feature requests, enhancements, documentation needs, or general project tasks.
- Issues can be assigned labels - *'good first issues', 'bug', 'enhancement'*.
- Other features -
 - Assignees
 - Comments
 - References



FORKING



Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk ().*

Owner *

AnshumaanMishra

Repository name *

reactplusplus

✔ reactplusplus is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description (optional)

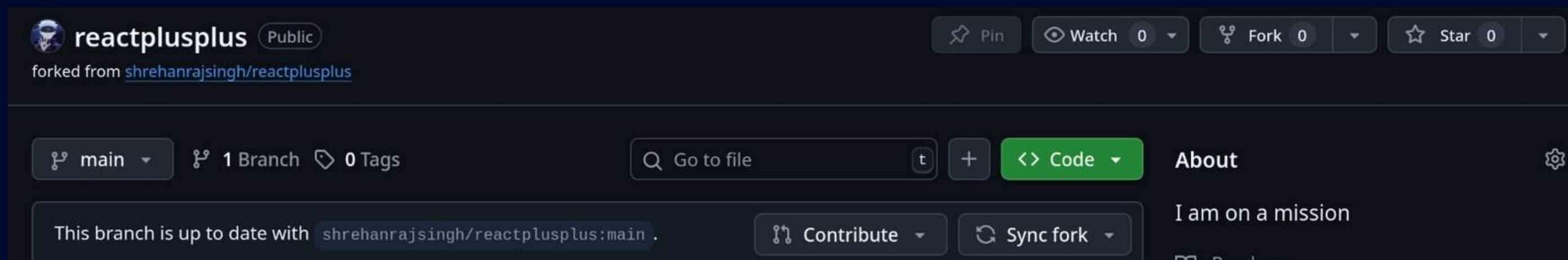
I am on a mission

☒ Copy the `main` branch only
Contribute back to shrehanrajsingh/reactplusplus by adding your own branch. [Learn more.](#)

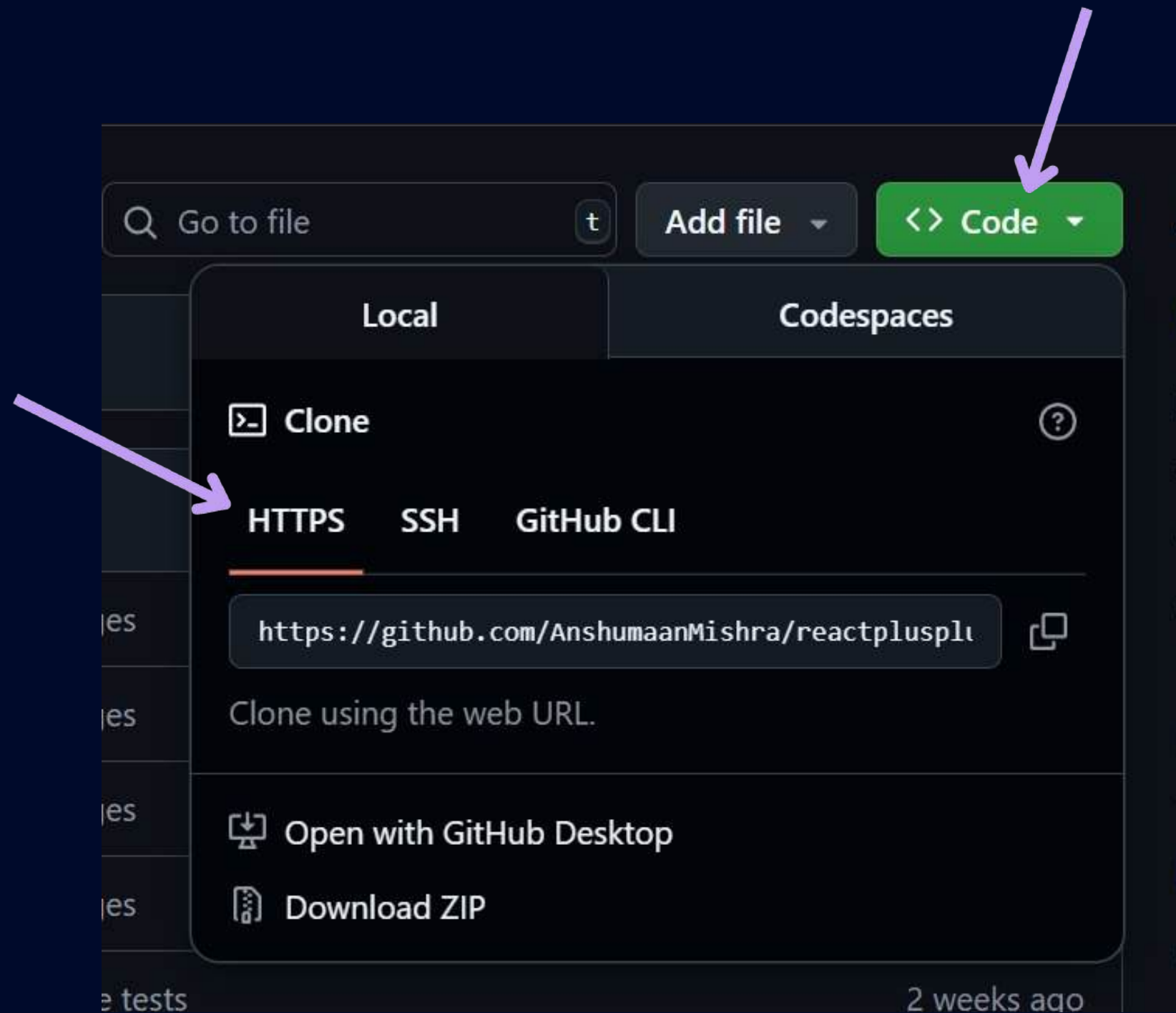
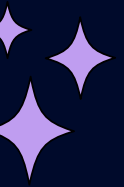
ⓘ

 You are creating a fork in your personal account.

Create fork



CLONING



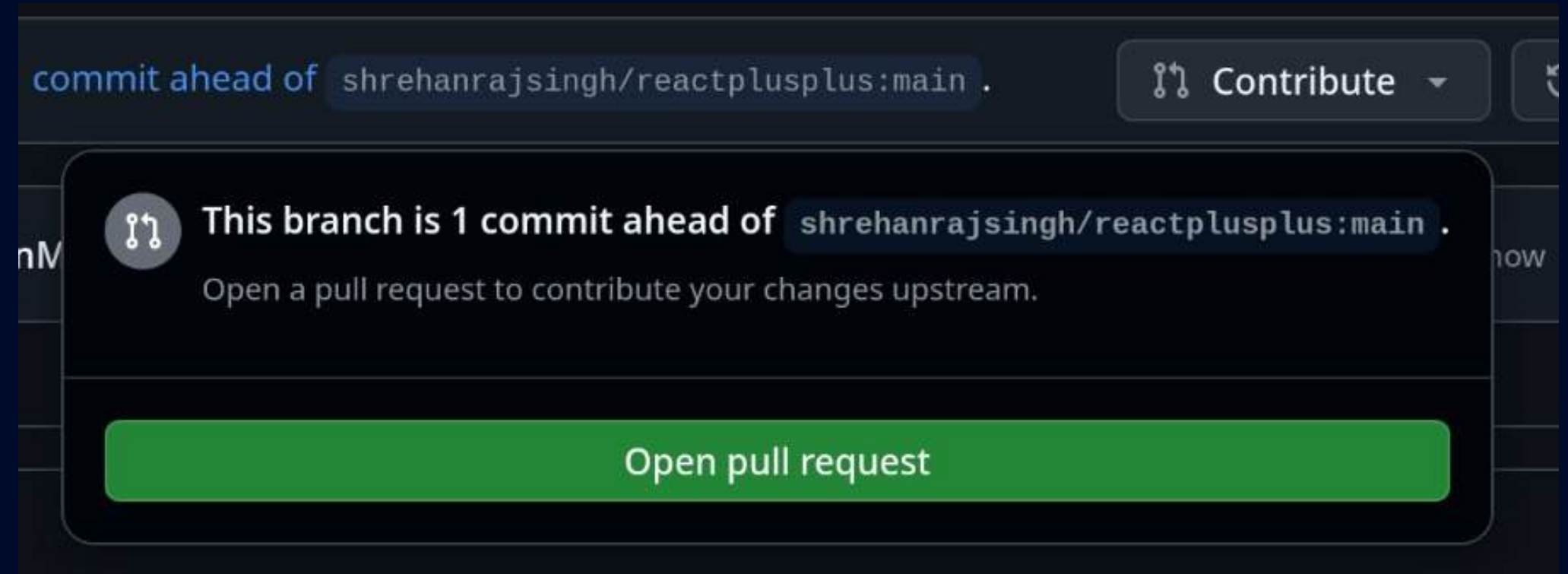
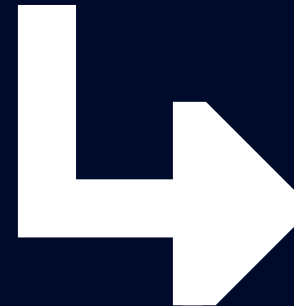
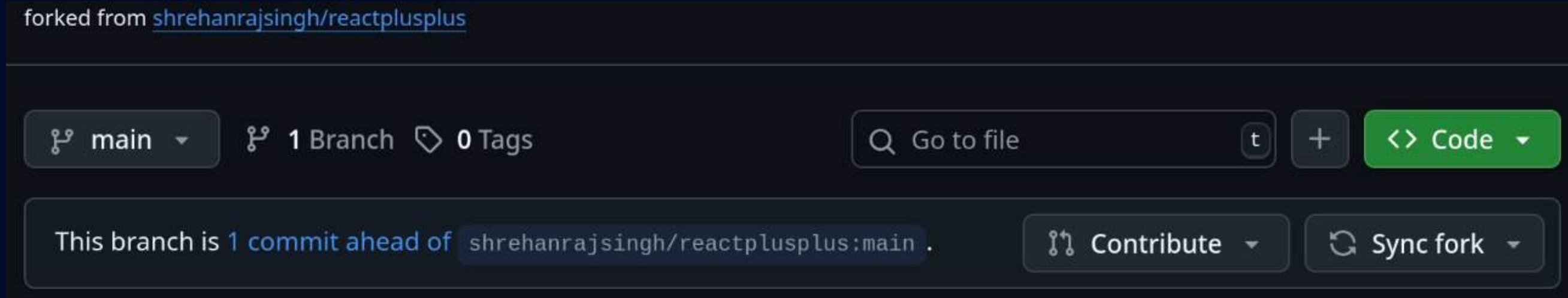
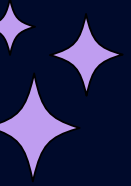
```
$ git clone <url>
```

Clones (downloads) a repository from GitHub to your computer.

A directory will be created with the same name as the repo. So to open the repo in your terminal, use

```
$ cd <repo-name>
```

PULL REQUESTS



PULL REQUESTS



Give your PR a
descriptive title

The screenshot shows a GitHub Pull Request interface. At the top, the title is "fix: I am doing the world a favour #1". Below the title, it says "AnshumaanMis... wants to merge 1 commit into shrehanrajsingh:main from AnshumaanMishra:main". There are buttons for "Open", "Edit", and "Code". Below this, there are tabs for "Conversation" (0), "Commits" (1), "Checks" (0), and "Files changed" (23). The "Files changed" tab is selected, showing a diff with a red bar indicating 4,341 lines changed. Below the diff, there is a comment by "AnshumaanMishra" that says "No description provided." To the right of the comment, there is a section for "Reviewers" which says "No reviews" and "Still in progress? Convert to draft".

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base repository: shrehanrajsingh/reactplus-plus base: main head repository: AnshumaanMishra/reactplus... compare: main



Add a title

fix: I am doing the world a favour

Helpful resources

[GitHub Community Guidelines](#)

Add a description

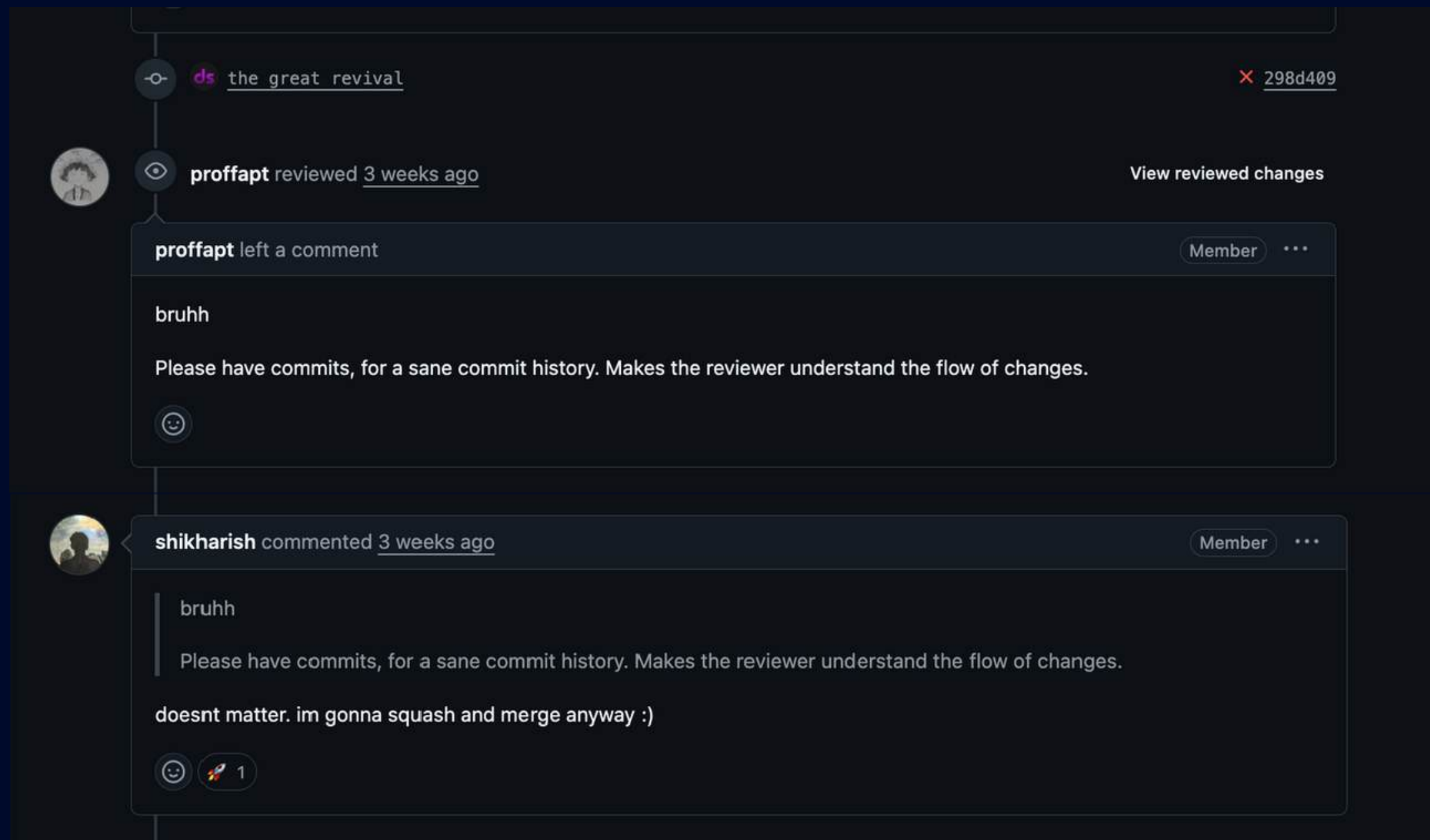
Write

Preview

H B I $\equiv$ <>

In the description, describe what your
PR does, what issue it fixes, etc.

- Your PR will get discussed in the comments
- Maintainers will review your PR and sometimes request for changes.
- Maintainers will merge your PR once it has approved.



SIDE QUEST 3

1. Fork <https://tinyurl.com/s8vzhnxx> 3 points
2. Set remote origin to this fork 3 points
3. Push all the commits to it 3 points
4. Add a file of your choice and
put something you want to share with others 1 points
(don't put passwords or other secrets)
5. Add and commit the changes with a message 2 points
6. Push all the commits to the fork 3 points
7. Create a pull request (with description) 5 points

PS: use `git status` in between commands to make sure things are working correctly, use `git log` to view the commit history

LIST OF COMMANDS



<code>git init</code>	Initialises a git repository in the current directory
<code>git clone <url></code>	Clones a GitHub repository on your computer
<code>git add .</code>	Adds the changes made to the 'staging area' so they are ready for committing
<code>git commit -m "msg"</code>	Creates a commit with the added changes
<code>git status</code>	Shows the current status of the repo
<code>git push origin <branch></code>	Pushes the new commits in <branch> to GitHub
<code>git branch</code>	Shows a list of branches and the current branch
<code>git branch <branch></code>	Creates a new branch called <branch>
<code>git checkout <branch></code>	Switch to the branch <branch>

DO IT YOURSELF



- Search for `kossiitkgp/sandbox` on GitHub
- Fork the repository to your GitHub account
- Clone your fork to your computer
- Create a new branch named `update-info`, switch to that branch
- Create a file called `<your-github-username>.txt`
- Write some message in the file
- Add and Commit your changes and push it to GitHub
- Create a Pull Request from your `update-info` branch to the main repo

GIT GOOD :)

- learngitbranching.js.org
- ohmygit.org
- [Git cheatsheet](#)
- [Build your own Git](#)

MAKE YOUR FIRST CONTRIBUTION

- [Check out metaKGP repositories](#)
- Start with issues labelled as *Good first issue*
- goodfirstissue.dev
- Participate in KWOC 2026 (Follow KOSS on [socials](#))



WE NEED YOUR FEEDBACK!

tinyurl.com/git-github-26-feedback



SLIDES:



FOLLOW OUR WHATSAPP CHANNEL

tinyurl.com/koss-whatsapp

